

Festival

Nayra is op een festival een spel aan het spelen, waar de hoofdprijs een trip naar Laguna Colorada is. Het spel bestaat uit tokens waarmee je coupons kan kopen. Het kopen van een coupon kan extra tokens opleveren. Het doel is om zo veel mogelijk coupons te kopen.

Ze start het spel met A tokens. Er zijn N coupons, genummerd van 0 tot en met $N - 1$. Nayra moet $P[i]$ tokens ($0 \leq i < N$) betalen om coupon i te kopen (en ze moet tenminste $P[i]$ tokens hebben voor de aankoop). Ze kan elke coupon maximaal één keer kopen.

Daarnaast krijgt elke coupon i ($0 \leq i < N$) een **type**, aangegeven met $T[i]$, wat een geheel getal **tussen de 1 en 4 is, inclusief**. Nadat Nayra coupon i koopt worden de overgebleven tokens vermenigvuldigd met $T[i]$. Formeel, als ze op een punt in het spel X tokens heeft en coupon i koopt (wat vereist dat $X \geq P[i]$), dan heeft ze na de aankoop $(X - P[i]) \cdot T[i]$ tokens over.

Jouw taak is om te bepalen welke coupons Nayra moet kopen en in welke volgorde om het aantal **coupons** aan het einde te maximaliseren. Als er meerdere manieren zijn die een maximaal aantal coupons geven, mag je een daarvan teruggeven.

Implementatie details

Je moet de volgende procedure implementeren.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A : het initiële aantal tokens dat Nayra heeft.
- P : een array met lengte N waarin de prijzen van de coupons staan.
- T : een array met lengte N waarin de types van de coupons staan.
- Deze procedure wordt exact één keer per testgeval aangeroepen.

De procedure moet een array R teruggeven met daarin de aankopen van Nayra op de volgende manier aangegeven:

- De lengte van R moet het maximum aantal coupons dat ze kan kopen zijn.
- De elementen van de array moeten de indexen van de coupons zijn die ze koopt, in chronologische volgorde. Dat wil zeggen dat ze coupon $R[0]$ eerst koopt, coupon $R[1]$ als tweede, enzovoorts.

- Alle elementen van R moeten uniek zijn.

Als er geen coupons kunnen worden gekocht, moet R een lege array zijn.

Beperkingen

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ voor elke i met $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ voor elke i met $0 \leq i < N$.

Subtaken

Subtaak	Score	Extra beperkingen
1	5	$T[i] = 1$ voor elke i met $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ voor elke i met $0 \leq i < N$.
3	12	$T[i] \leq 2$ voor elke i met $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Nayra kan alle N coupons kopen (in zekere volgorde)
6	16	$(A - P[i]) \cdot T[i] < A$ voor elke i met $0 \leq i < N$.
7	18	Geen extra beperkingen.

Voorbeelden

Voorbeeld 1

Beschouw de volgende aanroep.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Nayra had initieel $A = 13$ tokens. Ze kan 3 coupons kopen in de volgorde hieronder:

Coupon gekocht	Coupon prijs	Coupon type	Aantal tokens na de aankoop
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

In dit voorbeeld is het niet mogelijk voor Nayra om meer dan 3 coupons te kopen en is de volgorde van aankopen hierboven beschreven de enige manier waarop ze er 3 kan kopen. Dus moet de procedure $[2, 3, 0]$ teruggeven.

Voorbeeld 2

Beschouw de volgende aanroep.

```
max_coupons(9, [6, 5], [2, 3])
```

In dit voorbeeld kan Nayra beide coupons in elke volgorde kopen. Dus moet de procedure $[0, 1]$ of $[1, 0]$ teruggeven.

Voorbeeld 3

Beschouw de volgende aanroep.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

In dit voorbeeld heeft Nayra één token, wat niet genoeg is om een coupon te kopen. Dus moet de procedure $[]$ teruggeven (een lege array).

Voorbeeld Grader

Input format:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Output format:

```
S
R[0] R[1] ... R[S-1]
```

Hier is S de lengte van de array R teruggegeven door `max_coupons`.